The Conference Room Scheduler

A DSA Project: Leveraging Advanced Data Structures for Conflict-Free Booking

The Problem: Manual Booking Conflicts

Organizations and universities manage numerous conference rooms. Manual scheduling of these rooms by time slot often leads to **conflicts, double-bookings, and inefficient resource allocation.**

The Solution: Data Structures in Action

We use a specialized data structure to manage time slots, ensuring real-time conflict checking and high-speed operation, demonstrating how DSA improves real-world tasks.

Made with GAMMA

The Fixed Universe Successor Problem

This section introduces the Fixed Universe Successor Problem, a fundamental challenge in computer science, and explains how the Van Emde Boas (VEB) tree provides an efficient solution. The VEB tree is a **cool data structure** created to solve this problem, specifically for **finding elements after the given element in sorted order**.

- ## The Problem

  We aim to efficiently store and retrieve a dynamic subset S from a large, fixed universal set U.

- ## Universal Set U

  The universal set $U = \{0, 1, 2, ..., u - 1\}$ is a dynamic set, representing all possible elements in our domain.

- ## Dynamic Subset S

  A dynamic subset $S \subseteq U$ contains elements currently present, where elements can be inserted or deleted.

- ## The Goal

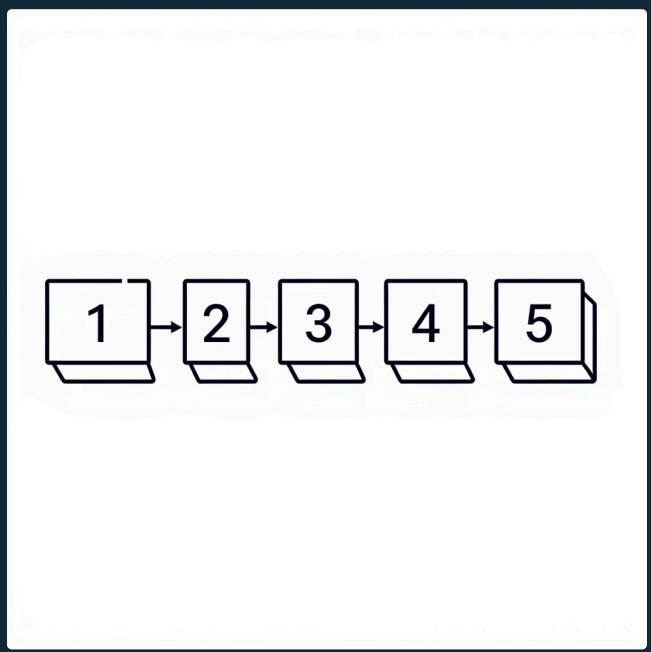
  Given an element $x \in U$, the objective is to find its successor, i.e., the smallest element $y \in S$ such that $y \geq x$.

Made with GAMMA

Why VEB Trees? A Quick Comparison

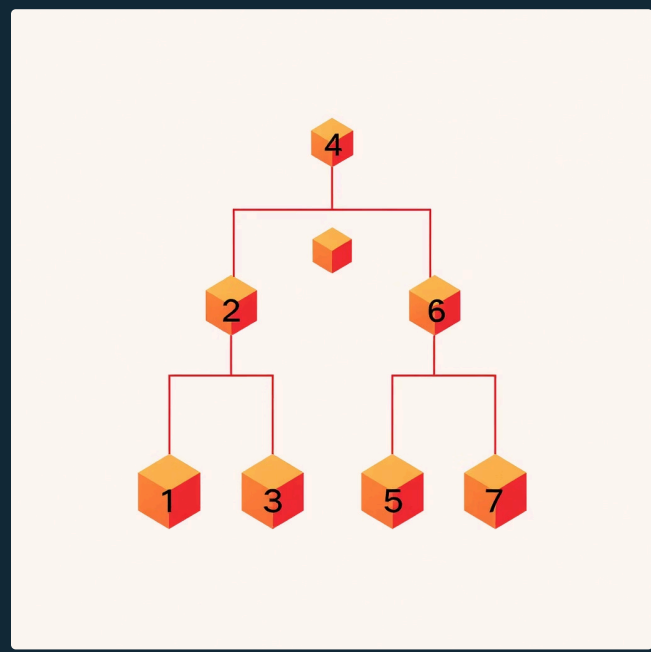
Understanding the strengths of VEB trees requires comparing them to more traditional data structures. Each excels in different scenarios, but VEB trees offer unparalleled performance for specific operations in a dense universe.

Linked List



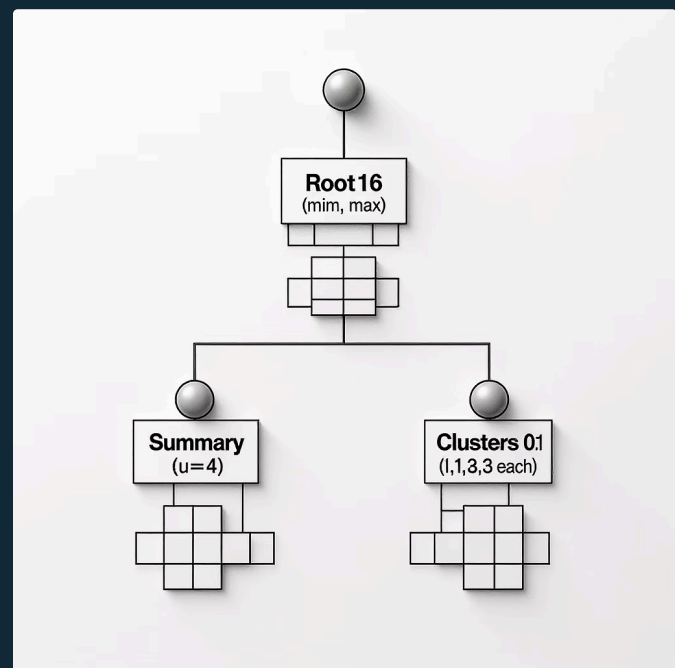
Operations: Search O(N), Insertion/Deletion O(1) (if position known), O(N) (if searching first).
Use Case: Simple, dynamic lists where element order and frequent insertions/deletions at known positions are key, but search speed is not critical.

Balanced Binary Tree



Operations: Search, Insertion, Deletion all O(log N).
Use Use Case: General-purpose data storage where all operations need to be efficient on dynamic data, maintaining logarithmic time complexity.

Van Emde Boas (VEB) Tree



Operations: Search (successor/predecessor), Insertion, Deletion all O(log log U), where U is the universe size.
Use Case: Optimal for sparse subsets of a dense, known universe where operations like finding the next available slot need ultra-fast performance.

Made with GAMMA

VEB Trees: Recursive Structure and Bit Representation

The efficiency of Van Emde Boas (VEB) trees stems from their recursive structure, which allows operations to run in $O(\log \log U)$ time, where U is the size of the universe. This is defined by the following recursive relation:

$$T(u) = T(\sqrt{u}) + O(1)$$

For illustrative purposes, let's consider a universe size of **u=16**. According to the recursive relation, we would then consider $\sqrt{u}$ **= 4** for the subproblems.

Bit Vector Representation (u=16)

u={0,1,2,3,4,5.....15} s={3,5,6,8}

1 0 1 0 1 0 1 1 0 1 1 1 0 1 0 1 0 1 0 1 0 1

VEB Tree Structure for u=16

Root (u=16) [min=?, max=?]

/ | | \

cluster0 cluster1 cluster2 cluster3 (each cluster u=4)

(u=4) (u=4) (u=4) (u=4)

. . . .

summary (u=4) ← stores which clusters (0..3) are non-empty.

Made with GAMMA

VEB Trees: Practical Applications

Key Applications

- **Dynamic Ordered Sets:** Swift predecessor/successor queries on integer ranges.
- **Priority Queues:** Efficient integer-keyed priority management within bounded ranges.
- **Network Algorithms:** Fast neighbor queries in routing and resource allocation.
- **Computational Geometry:** Integer-keyed event handling in simulations and geometric data structures.

Why VEB for Our Project?

- **Fast `find_min()`:** Quickly identify the earliest available booking slot.
- **Successor/Predecessor:** `O(log log U)` speed for finding next/previous bookings.
- **Rapid Range Queries:** Instantly locate the next available or conflicting slot, critical for conflict-free scheduling.

Made with GAMMA

VEB Tree Bit Vector Operations

For a universe size of $u = 16$, the square root is $\sqrt{u} = 4$. This means we can divide our 16-bit universe into 4 clusters, each containing 4 bits.

Let's use the bit vector representation for $u = 16$, with elements $s = \{3, 5, 6, 9\}$ currently present. The bits are set at positions 3, 5, 6, and 9.

```
[0 | 0 | 0 | 1] [0 | 1 | 1 | 0] [0 | 1 | 0 | 0] [0 | 0 | 0 | 0]
Bit Position:
0 1 2 3         4 5 6 7         8 9 10 11       12 13 14 15
```

Adding Element 7

To add an element, say $x = 7$, we first determine its cluster and position within that cluster.

- Binary representation of $x = 7$ is `0111`.
- The high part, $h(x)$, determines the cluster. For 7, `01` (binary for 1) means Cluster 1.
- The low part, $l(x)$, determines the bit position within the cluster. For 7, `11` (binary for 3) means position 3 within Cluster 1.

To insert 7, we set the bit at `block[h(7)][l(7)] = block[1][3]` to 1.

```
[0 | 0 | 0 | 1] [0 | 1 | 1 | 1] [0 | 1 | 0 | 0] [0 | 0 | 0 | 0]
```

Deleting Element 7

To delete an element, for example, the element 7 we just added, we simply reverse the insertion process.

- Using $h(7) = 1$ and $l(7) = 3$, we target `block[1][3]`.

To delete 7, we set the bit at `block[h(7)][l(7)] = block[1][3]` to 0.

```
[0 | 0 | 0 | 1] [0 | 1 | 1 | 0] [0 | 1 | 0 | 0] [0 | 0 | 0 | 0]
```

Made with GAMMA

The Scheduler in Action: Conflict-Free Logic

How VEB Trees Power Our Conference Scheduler

The system maps each room/slot combination to a unique integer index: `index = (room-1) * slots + (slot-1)`.

10
Rooms

15
Slots Per Room

150
Total Slots

16
VEB Size (Power of 2)

All primary booking functions leverage the VEB Tree for speed:

Add Booking	Manual/Smart Booking	O(log log U)
Remove Booking	Delete Booking	O(log log U)
Check Availability	is_member()	O(log log U)
First Free Slot	VEB min (Inverted)	O(log log U) (If optimized for successor)

Made with GAMMA

Admin Visibility: Summaries and Visual Status

Functions to Provide Instant Feedback to Administrators

1. Availability Summary

Provides a quick snapshot of resource utilization.

- Total Rooms: 10
- Slots per Room: 15
- **Booked Slots: 30**
- **Free Slots: 120**

Helps the admin see how full the schedule is at a glance.

2. View All Bookings

Lists all current bookings in an ordered, readable format.

- 1. Room 1, Slot 2
- 2. Room 1, Slot 3
- 3. Room 2, Slot 5

The indices from the VEB tree are converted back to meaningful room and slot numbers.

---

3. View Room Status (Visual Dashboard)

A visual grid showing status for all 150 slots.

Goal: Instant, intuitive understanding of room availability.
(e.g., Room 3: Slot 1-5 ❌❌❌✅✅ Slot 6-10 ✅✅❌✅✅)

Made with GAMMA

Debugging & Conclusion

Visualizing the VEB Tree and User Interface

VEB Tree Visualization

The `visualize_veb_tree()` function recursively prints nodes to the console, showing:

- Min/Max values at each node.
- The universe size (`u`).
- The depth of the recursion.

```
[0, 127] (u=128)
├─── [0, 63] (u=64)
└─── [64, 127] (u=64)
```

This tool is crucial for debugging and understanding how bookings are clustered.

User Interaction Layer

The console interface provides the administrative menu, connecting user commands to the core `ConferenceScheduler` logic.

- Smart Booking
- Manual Booking
- Delete Booking
- View Schedule

Made with GAMMA